

Visualize IoT in Real-Time

Build Live Dashboard with AWS in Minutes



Apil Chaudhary

AWS Certified Solutions Architect
Event Lead | AWS CC IoE Pulchowk

Scenario: Smart Farm

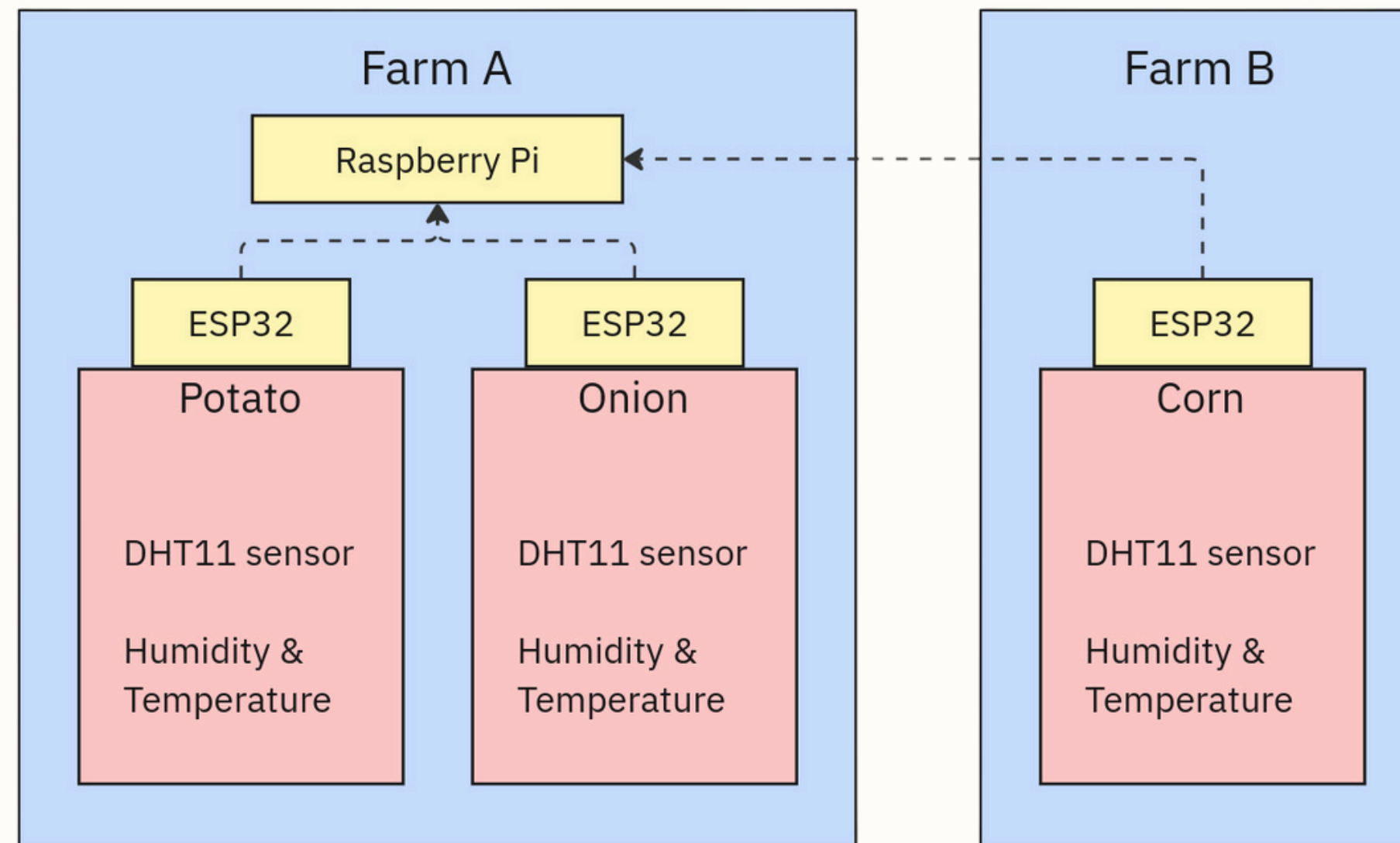


Fig. Setup of Our Smart Farm

Architecture

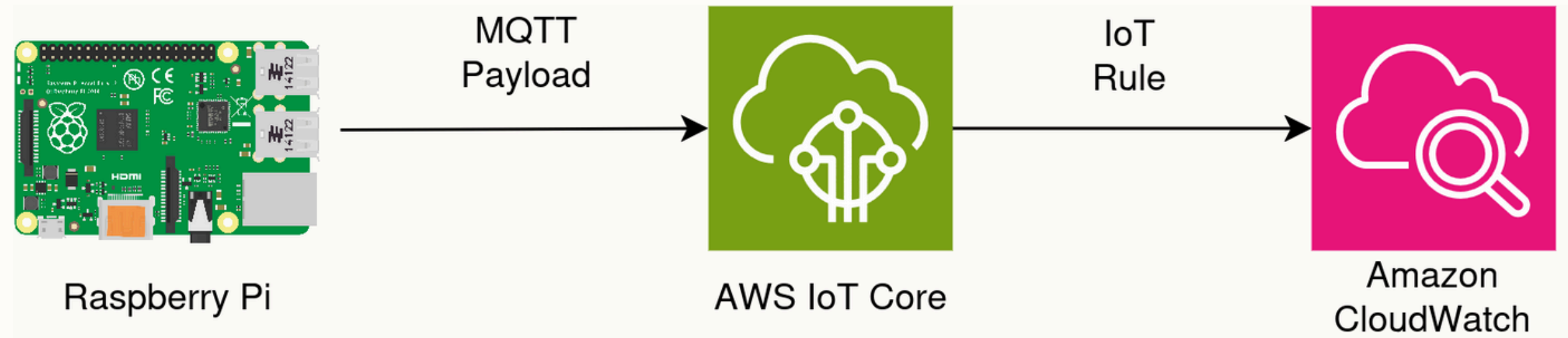


Fig. AWS Architecture

Raspberry Pi

Publishes Sensor Data to a Topic

AWS IoT Core

Catches the Data & Send to
CloudWatch Metrics

Amazon CloudWatch

Visualizes the Received Metrics

IoT Thing

- Like a “***Digital Twin***”
- Connection kit provides us with ***Device Certificate, Private Key, Startup Script***
- The script downloads ***root-CA Certificate, AWS-IoT-SDK repo***

Connection kit

Certificate FarmA_Potato.cert.pem	Private key FarmA_Potato.private.key	AWS IoT Device SDK Python
Script to send and receive messages start.sh	Policy FarmA_Potato-Policy View policy	

Download

If you are running this from a browser on the device, after you download the connection kit, it will be in the browser's download folder.

If you are not running this from a browser on your device, you'll need to transfer the connection kit from your browser's download folder to your device using the method you tested when you prepared your device in step 1.

[Download connection kit](#)

Fig. Connection Kit to connect from AWS IoT SDK

Automatic Run Test

```
Publishing message to topic 'sdk/test/python': Hello from mqtt5 sample [4]
PubAck received with <PubackReasonCode.SUCCESS: 0>

==== Received message from topic 'sdk/test/python': Hello from mqtt5 sample [4] ====

Publishing message to topic 'sdk/test/python': Hello from mqtt5 sample [5]
PubAck received with <PubackReasonCode.SUCCESS: 0>

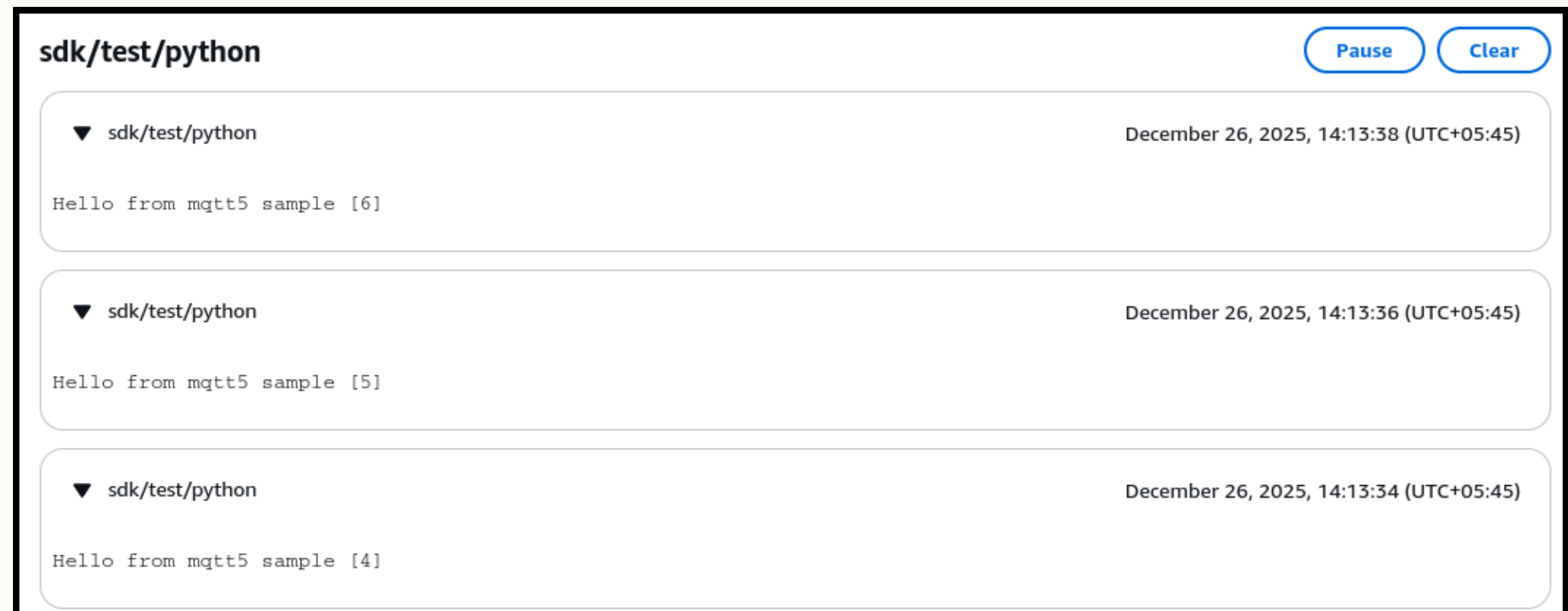
==== Received message from topic 'sdk/test/python': Hello from mqtt5 sample [5] ====

Publishing message to topic 'sdk/test/python': Hello from mqtt5 sample [6]
PubAck received with <PubackReasonCode.SUCCESS: 0>

==== Received message from topic 'sdk/test/python': Hello from mqtt5 sample [6] ====
```

*Fig. MQTT Client from
Local Linux Platform*

*Fig. MQTT test from
AWS Management Console*



Logical Connection

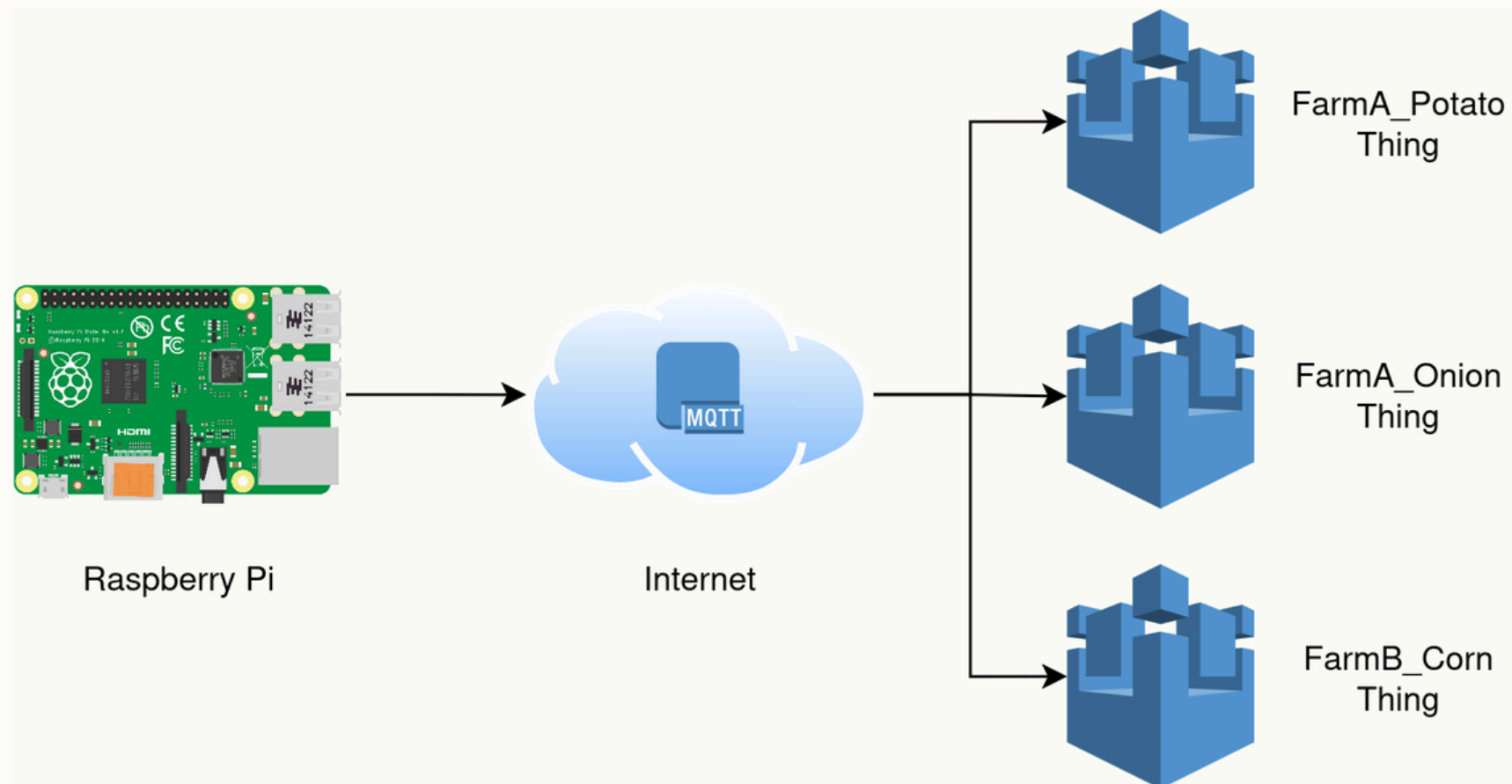


Fig. Farm Connection with IoT Things

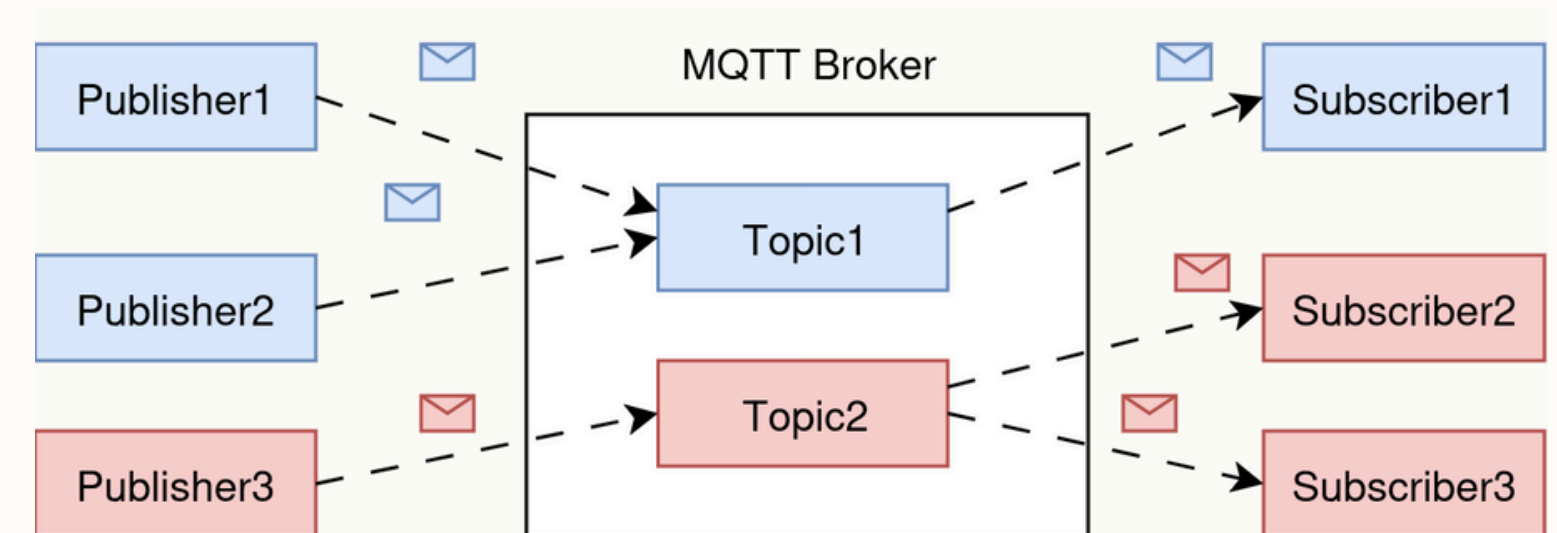


Fig. MQTT Pub/Sub Model

Policy Update

```
{
  "Effect": "Allow",
  "Action": [
    "iot:Publish",
    "iot:Receive",
    "iot:PublishRetain"
  ],
  "Resource": [
    "arn:aws:iot:<REG-ID>:<ACC-ID>:topic/<TOPIC>"
  ]
}
```

```
{
  "Effect": "Allow",
  "Action": "iot:Subscribe",
  "Resource": [
    "arn:aws:iot:<REG-ID>:<ACC-ID>:topicfilter/<TOPIC>"
  ]
}
```

```
{
  "Effect": "Allow",
  "Action": "iot:Connect",
  "Resource": [
    "arn:aws:iot:<REG-ID>:<ACC-ID>:client/<CLIENT>"
  ]
}
```

Fig. Add Permissions to Publish to **Custom Topics** and Create **Custom Clients**

Message Received (Halfway of the Journey)

```
# MQTT client
client_id = f"{args.farm_id}_{args.crop_name}"
client = mqtt5_client_builder.mtls_from_path(
    endpoint=args.endpoint,
    cert_filepath=args.cert,
    pri_key_filepath=args.key,
    client_id=client_id,
)
topic = f"farm/{args.farm_id}/{args.crop_name}/data"
```

Fig. MQTT Client Creation

```
# Inside publish loop
payload = json.dumps({"humidity": humidity, "temperature": temperature})
publish_future = client.publish(
    mqtt5.PublishPacket(
        topic=topic,
        payload=payload,
        qos=mqtt5.QoS.AT_LEAST_ONCE
    )
)
publish_future.result(TIMEOUT)
```

Fig. MQTT Payload Transmission

The screenshot displays two MQTT messages received by a test client in the AWS IoT console. The top message is from the topic `farms/farmA/potato/data` and was received at `December 26, 2025, 16:04:27 (UTC+0545)`. The payload is a JSON object: `{"humidity": 45.61608973555748, "temperature": 27.864710682508804}`. Below it, the 'Properties' section is visible. The bottom message is also from the topic `farms/farmA/potato/data` and was received at `December 26, 2025, 16:04:26 (UTC+0545)`. Its payload is a JSON object: `{"humidity": 43.229176374072956, "temperature": 25.65707712160919}`.

Fig. MQTT Test Client in AWS

Message Routing Rule

- **Only One Rule** : Use **wildcards** for sending **humidity & temperature** data from all topics i.e.

1. farms/farmA/potato/data
2. farms/farmA/onion/data
3. farms/farmB/corn/data

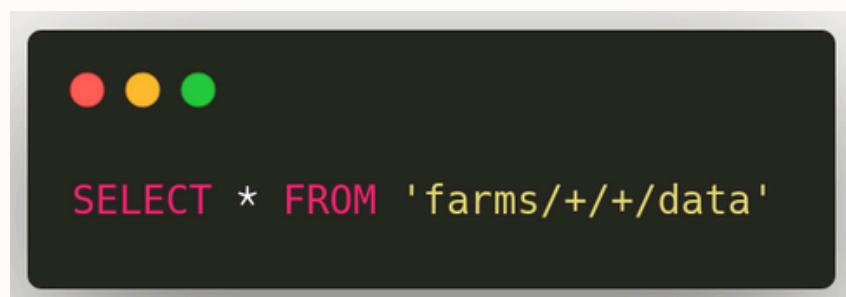


Fig. SQL Rule for Selecting Data to Route

Rule actions [Info](#)

Select one or more actions to happen when the above rule is matched by an inbound message. Actions define additional invoking cloud functions, or sending notifications. You can add up to 10 actions.

Action 1

▼ CloudWatch metric
Send message data to CloudWatch metric

Metric name [Info](#)
Humidity

Metric namespace
SmartFarm

Unit
Percent

Value
\${humidity}

Timestamp - optional
The timestamp that will be added to the posted metric.

IAM role
Choose a role to grant AWS IoT access to your endpoint.
msgRouteRole [View](#) [Create new role](#)

AWS IoT will automatically create a policy with a prefix of "aws-iot-rule" under your IAM role selected.

Fig. Rule Actions to CloudWatch Metrics

CloudWatch Metrics

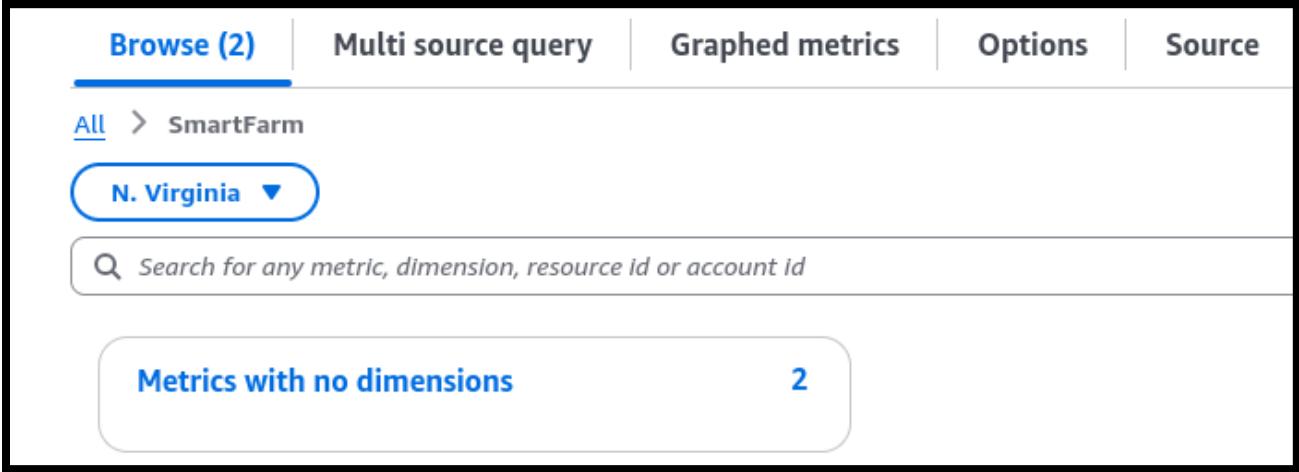


Fig. SmartFarm Namespace

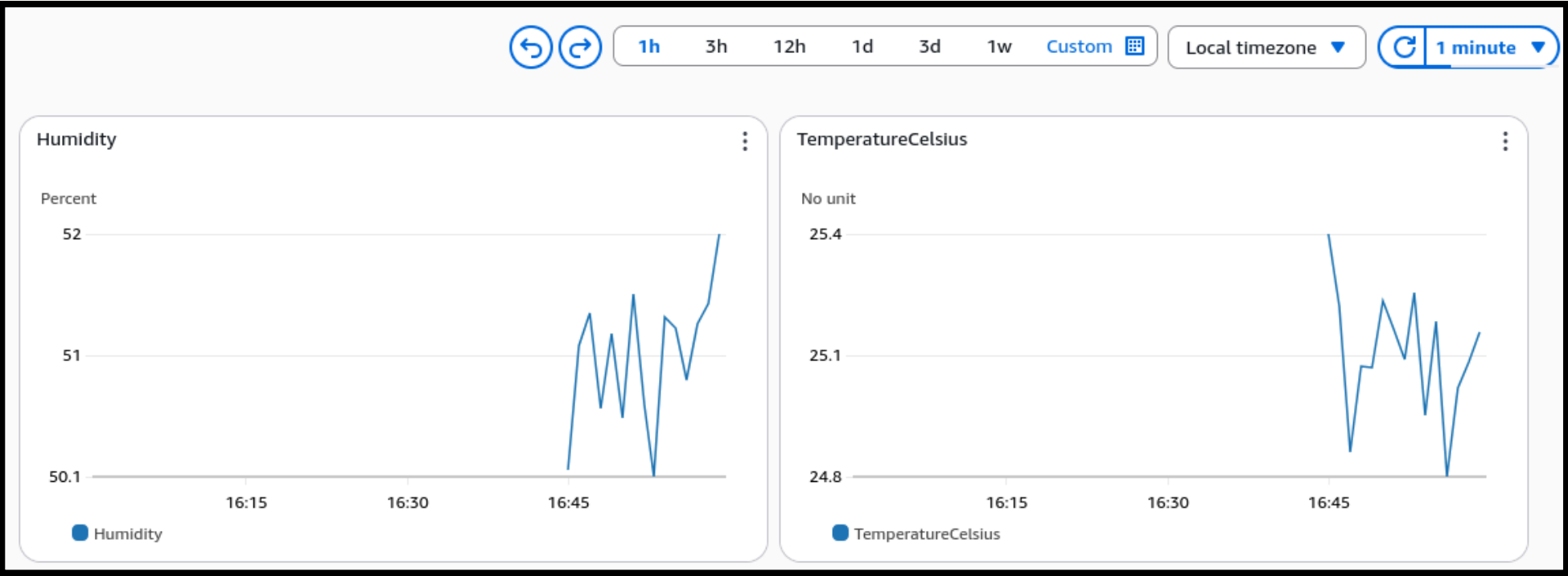


Fig. Custom CloudWatch Dashboard with Two Widgets

☒	Label	Details	Statistic	Period
☒	Humidity	Region: us-east-1 • SmartFarm • Humidity	Average ▼	1 minute ▼

Fig. Selecting Aggregation Type and Period for Metric

Our Implementation Failed!!!

All 3 IoT Things publishing same metrics, and
overall aggregation is being visualized!

What If, I just want to look:
only farms/farmB/corn/data
OR
aggregated data of farms/farmA
???

Dimensions - “Labels to the Metrics”

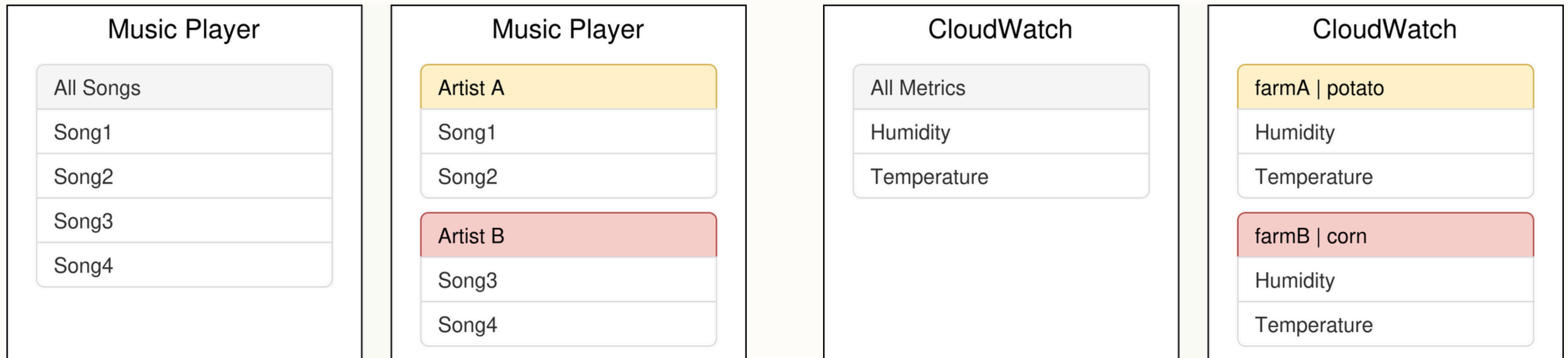


Fig. Music Player Analogy for CloudWatch Dimensions

Refined Architecture

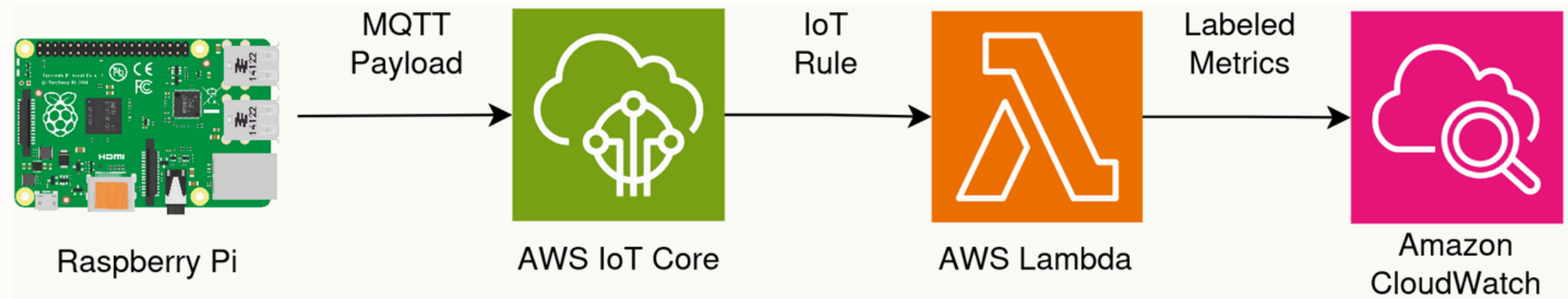


Fig. Refined AWS Architecture

- IoT Rule **does not allow** to add dimensions to CloudWatch metrics
- AWS Lambda is invoked to add dimension to metrics

Disclaimer: Don't forget to add **CloudWatch:PutMetricData** permissions to the Lambda Execution Role

Lambda Function Integration

farms / farmA / potato / data

Farm ID **Crop ID**

Fig. Select Dimensions from Topic Name

```
SELECT *, topic(2) as farm_id,  
topic(3) as crop_id FROM 'farms/+/+/data'
```

Fig. Extract Fields for Dimensions

```
cloudwatch.put_metric_data(  
    Namespace="SmartFarm",  
    MetricData=[  
        {  
            "MetricName": "Humidity",  
            "Dimensions": [  
                {"Name": "Farm ID", "Value": farm_id},  
                {"Name": "Crop ID", "Value": crop_id},  
            ],  
            "Unit": "Percent",  
            "Value": float(humidity),  
        },  
        {  
            "MetricName": "Temperature",  
            "Dimensions": [  
                {"Name": "Farm ID", "Value": farm_id},  
                {"Name": "Crop ID", "Value": crop_id},  
            ],  
            "Unit": "None",  
            "Value": float(temperature),  
        },  
    ],  
)
```

Fig. Lambda Function Snippet

Metrics with Dimensions

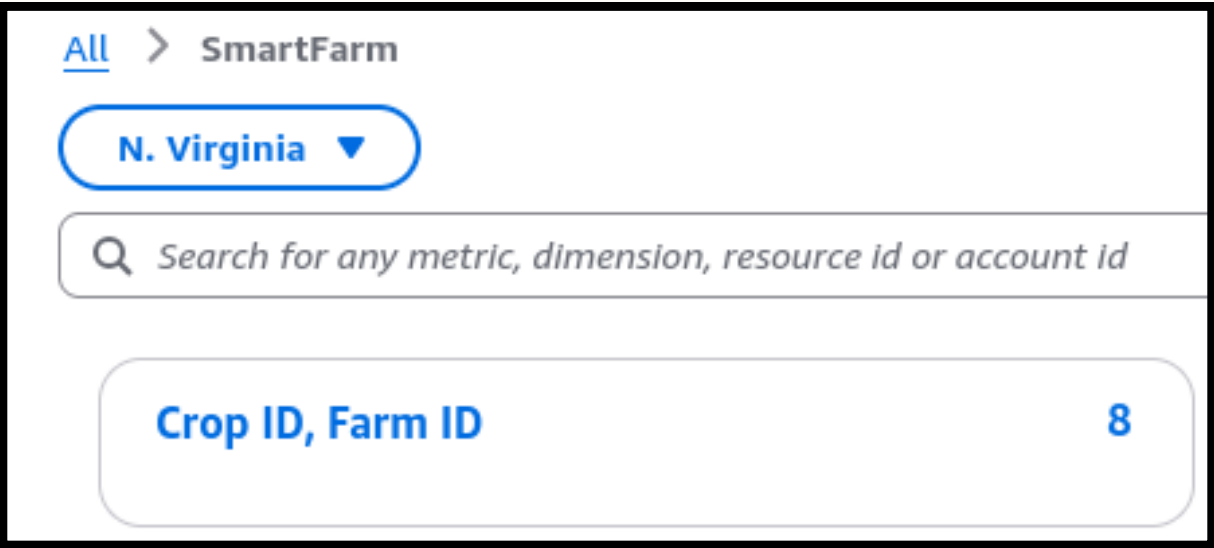


Fig. New SmartFarm Namespace

☐	Crop ID 8/8 ▲	Farm ID ▼	Metric name ▼
☐	corn	farmB	Temperature
☐	corn	farmB	Humidity
☐	onion	farmA	Temperature
☐	onion	farmA	Humidity
☐	potato	farmA	Temperature
☐	potato	farmA	Humidity

Fig. Metrics along with Dimensions

☒		ID ⓘ	Label	Details	Statistic	Period	Y axis
☒		e1 ⓘ	Expression1 ⓘ	AVG([m2,m3]) ⓘ			< >
☒		m1 ⓘ	corn farmB ⓘ	Region: us-east-1 • SmartFarm • Tempera ⓘ	Average ▼	1 minute ▼	< >
☒		m2 ⓘ	onion farmA ⓘ	Region: us-east-1 • SmartFarm • Tempera ⓘ	Average ▼	1 minute ▼	< >
☒		m3 ⓘ	potato farmA ⓘ	Region: us-east-1 • SmartFarm • Tempera ⓘ	Average ▼	1 minute ▼	< >

Fig. Line plot with 3 metrics and 1 expression

Labeled Metrics Visualization

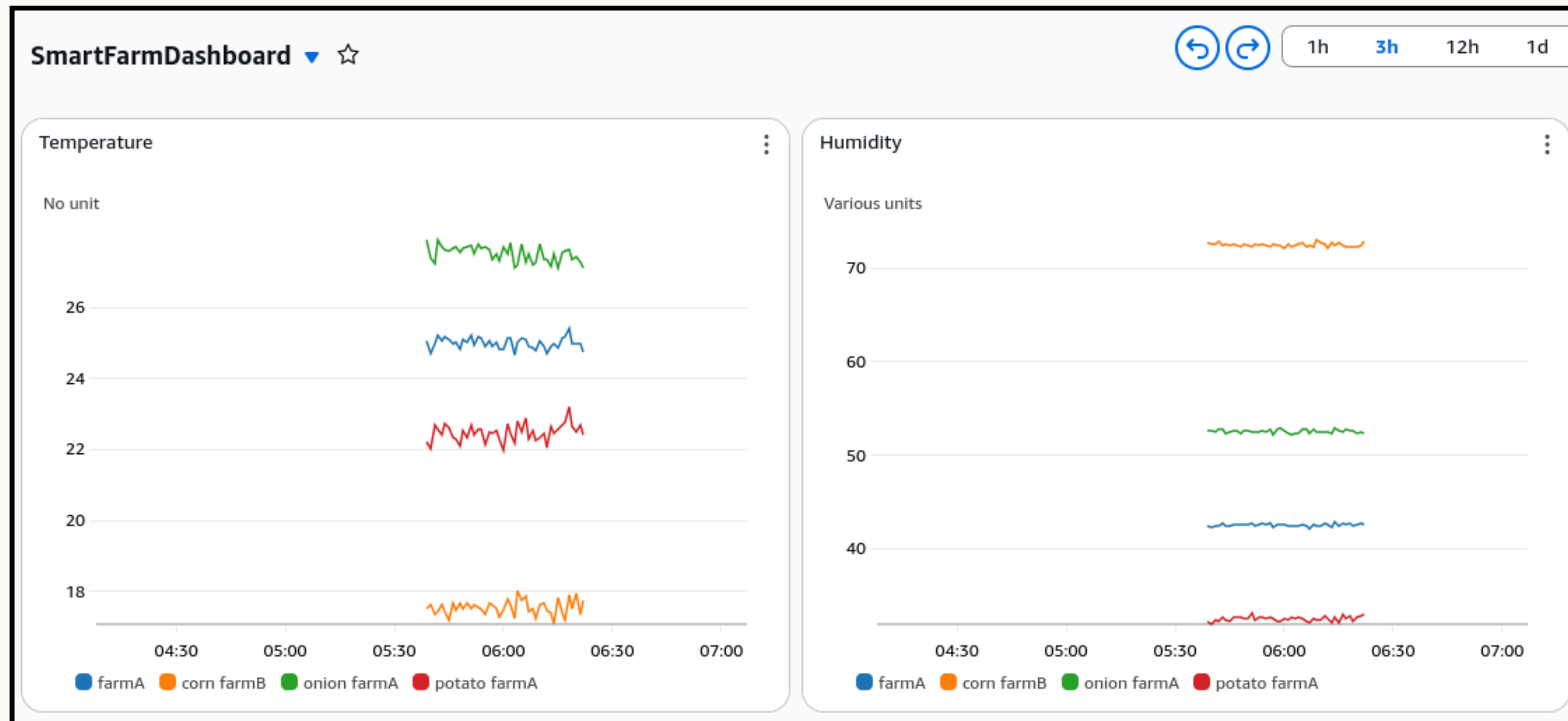


Fig. CloudWatch Custom Dashboard

Shareable Dashboard

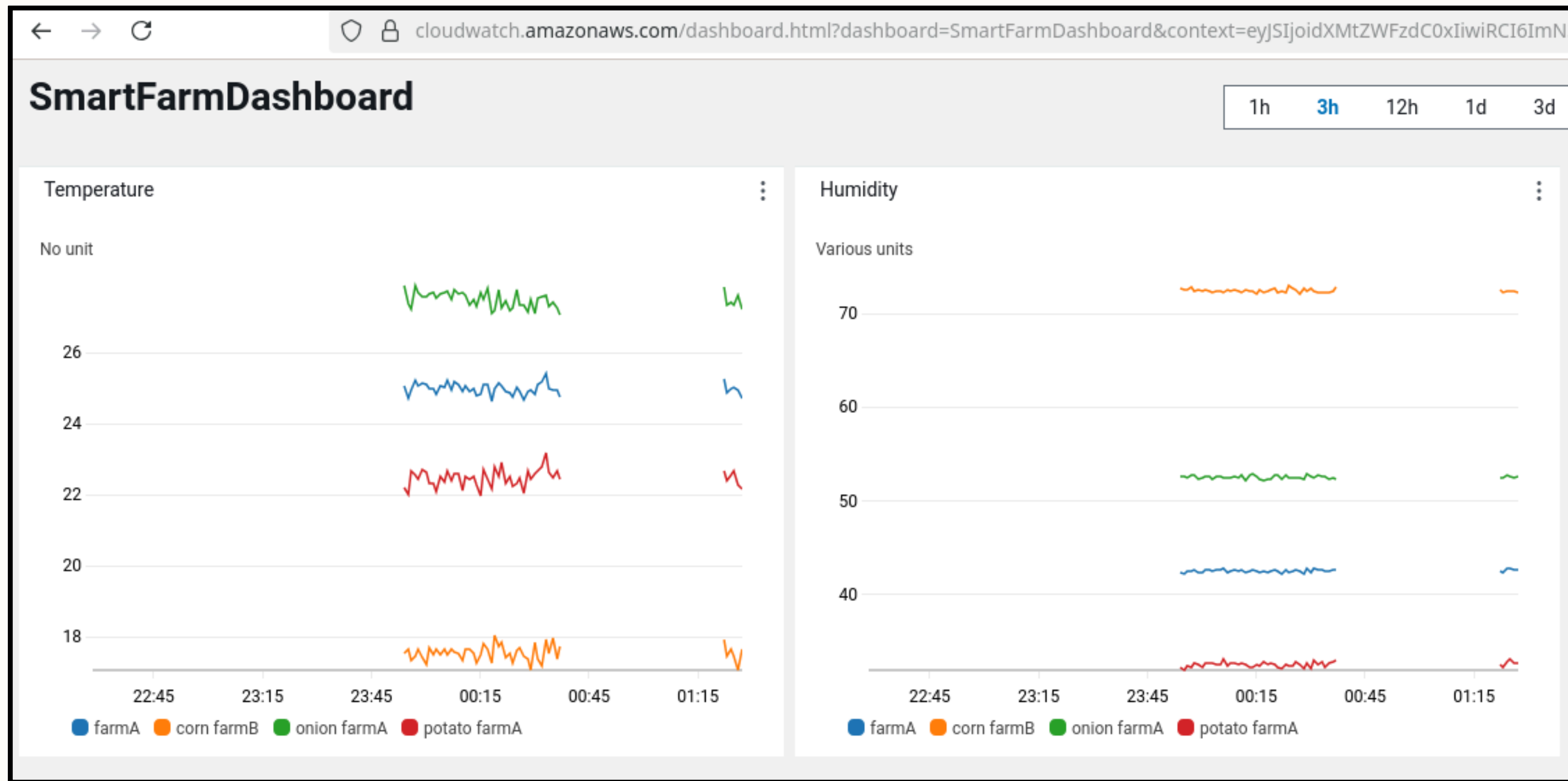


Fig. Shared Dashboard through Public URL

Stop Guessing, **Start Visualizing** !!!

World is full of data, Visualize/Analyze it

No Raspberry Pi? Run the script in your own computer

No Sensors? Visualize your own CPU metrics

Repo: ***github.com/tyro-apil/AWS_SCD_DEMO***

Connect with me: ***apilchaudhary.com.np***